The Trap - Shared Accumulator

- **Problem:** Calculate the Sum of all elements in A.
- **Serial:**

```
double sum = 0.0; for (int i=0; i<N; i++) sum += a[i];
```

Naive Parallel Attempt

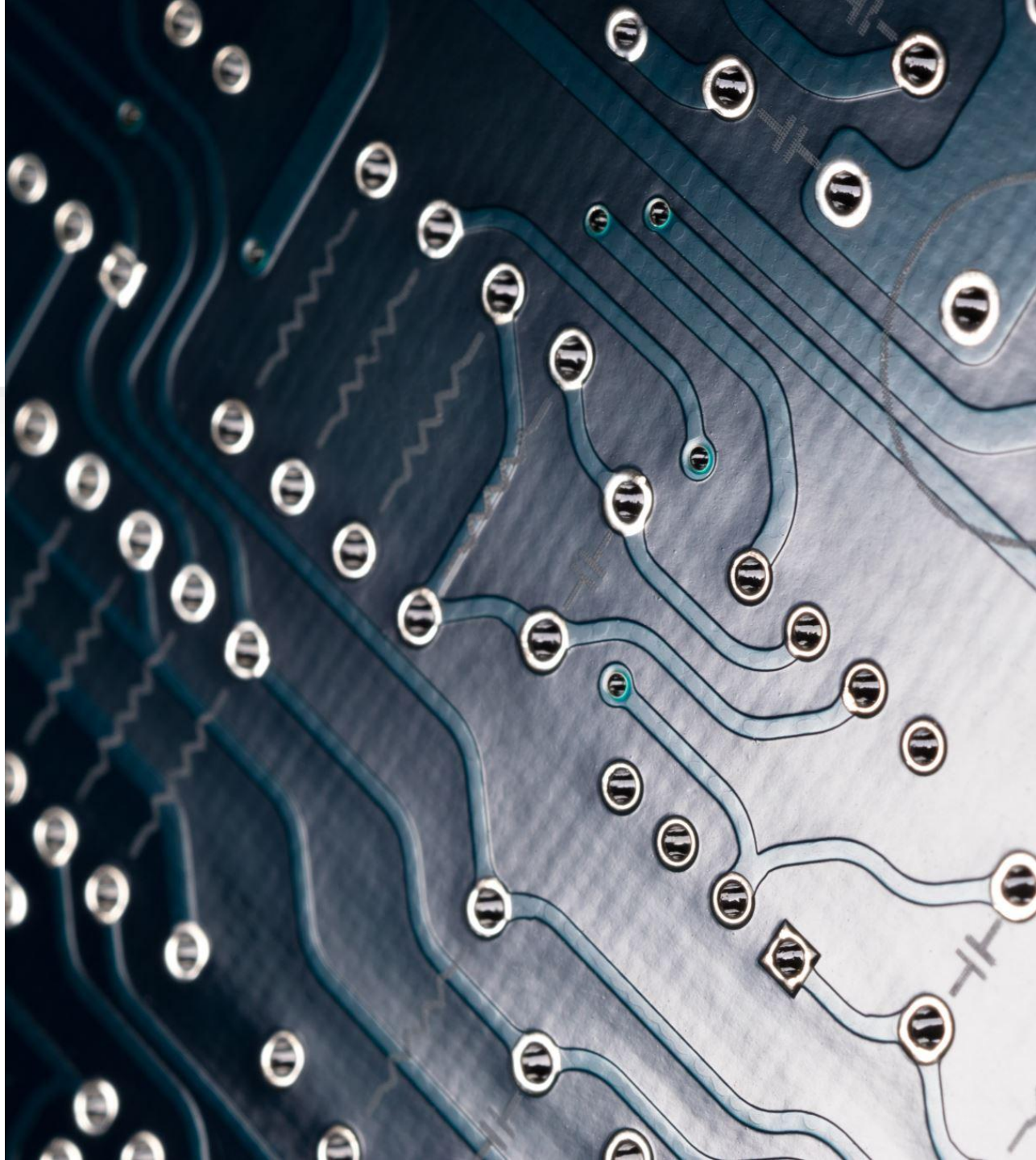
```
double sum = 0.0;
#pragma omp parallel
{
    // ... calculate start/end ...
    for (int i=start; i<end; i++) {
        sum += a[i]; // Danger!
    }
}
```

Data Race (The "Lost Update")

- Why sum += a[i] Fails in Parallel
  - The operation sum += a[i] looks like one step, but the CPU sees **three distinct steps**:
    - **READ** the current value of sum from memory into a register.
    - **ADD** a[i] to that register.
    - **WRITE** the new value back to memory.
  - **The Crash:** Two threads can **READ** the same old value (e.g., 10) at the same time. Both add their numbers (say, +1 and +2) and write back.
    - Thread A writes 11.
    - Thread B writes 12.
    - **Result:** The actual sum should be 13, but one update was overwritten and "lost."
  - **Outcome:** The final sum is unpredictable and usually smaller than the correct answer.

Definition: Race Condition

- Occurs when multiple threads access the same shared memory location.
- At least one access is a WRITE.
- The accesses are simultaneous (unsynchronized).
- **Result:** Corrupted data and unpredictable bugs.



Code 1: The Buggy Program

race.c

```c
int main() {
    long long sum = 0;
    int a[N];

    // Initialize array with 1s. Correct sum should be 1,000,000.
    for (int i = 0; i < N; i++) a[i] = 1;

    // --- THE BUGGY PARALLEL REGION ---
    #pragma omp parallel
    {
        // Manual work sharing
        int id = omp_get_thread_num();
        int nthr = omp_get_num_threads();
        int chunk = N / nthr;
        int start = id * chunk;
        int end = start + chunk;

        // FIX for remainder (from previous lesson)
        if (id == nthr - 1) end = N;

        for (int i = start; i < end; i++) {
            // CRITICAL BUG: Multiple threads update 'sum' simultaneously
            sum += a[i];
        }
    }

    printf("Actual Sum: %lld (Expected: %d)\n", sum, N);

    if (sum != N) printf("FAILURE: Race condition detected!\n");
    else printf("SUCCESS: (You got lucky, but the code is still unsafe)\n");

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./race
Actual Sum: 475799 (Expected: 1000000)
FAILURE: Race condition detected!
h8wcd@fcs-0028:~/openmp-demo$
```

Solution 1 – The Atomic Fix

- Protecting the Memory Address (#pragma omp atomic)
  - The atomic directive tells the hardware: "Do not let anyone else touch this specific memory address (sum) while I am updating it."
  - **Mechanism:** It locks the variable for the single instruction sum += ....
  - **Pros:** Very simple to add.
  - **Cons: Severe Performance Penalty.** If you have 1,000,000 iterations, you are engaging a hardware lock 1,000,000 times. This effectively serializes the memory access, often making the parallel code *slower* than serial code.

atomic_solution.c

```c
/* 2_atomic_solution.c */
#include <stdio.h>
#include <omp.h>

#define N 1000000

int main() {
    long long sum = 0;
    int a[N];

    for (int i = 0; i < N; i++) a[i] = 1;

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int nthr = omp_get_num_threads();
        int chunk = N / nthr;
        int start = id * chunk;
        int end = (id == nthr - 1) ? N : start + chunk;

        for (int i = start; i < end; i++) {
            // FIX: 'atomic' ensures only one thread updates 'sum' at a time
            #pragma omp atomic
            sum += a[i];
        }
    }

    printf("Atomic Sum: %lld (Expected: %d)\n", sum, N);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./race
h8wcd@fcs-0028:~/openmp-demo$ gcc -fopenmp atomic_solution.c -o atomic_solution
h8wcd@fcs-0028:~/openmp-demo$ ./atomic_solution
Atomic Sum: 1000000 (Expected: 1000000)
h8wcd@fcs-0028:~/openmp-demo$
```

Solution 1- better version (atomic parallel for)

atomic_parallel_for.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000

int main(void) {
    long long sum = 0;
    int *a = (int *)malloc((size_t)N * sizeof(int));
    if (!a) {
        perror("malloc failed");
        return 1;
    }

    // Initialize array with 1s. Correct sum should be N.
    for (int i = 0; i < N; i++) a[i] = 1;

    // Optional: set a fixed number of threads for a consistent demo
    omp_set_num_threads(4);

    // Automatic work-sharing + atomic update (correct but often slow)
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        #pragma omp atomic
        sum += a[i];
    }

    printf("Atomic Sum: %lld (Expected: %d)\n", sum, N);
    if (sum != N) printf("FAILURE: Race condition detected (should not happen with atomic)!\n");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ gcc -fopenmp atomic_parallel_for.c -o atomic_parallel_for
h8wcd@fcs-0028:~/openmp-demo$ ./atomic_parallel_for
Atomic Sum: 1000000 (Expected: 1000000)
h8wcd@fcs-0028:~/openmp-demo$
```

Solution 2 – Reduction (The Best Practice)

- Parallel Reduction (reduction(+:sum))
  - Instead of fighting over one shared sum, OpenMP gives every thread its **own private copy** of sum (initialized to 0).
  - **Phase 1 (Parallel):** Each thread adds its chunk of numbers to its *private* sum. No locks needed!
  - **Phase 2 (Merge):** When the threads finish, OpenMP secretly combines (reduces) all the private sums into the global sum.
  - **Pros:** extremely fast, scalable, and thread-safe.
  - **Cons:** Only works cleanly for “reducible” operations
    - +, *, min, max, bitwise &, |, ^
    - Examples that do **not** work well:
      - building a list in order
      - concatenating strings in a strict order
      - updating a complex structure with side effects



OpenMP reduction: Syntax

- **Goal:** Safely combine values computed by multiple threads into **one final result**.
- **Meaning**
  - Each thread gets a **private copy** of each reduction variable.
  - Each thread updates its private copy independently (no race condition).
  - At the end, OpenMP **merges** all private copies using op.

```
#pragma omp parallel for reduction(op: var1, var2, ...)
for (...) {
    // update var1, var2, ...
}
```

What Goes Inside reduction(…)

- **Format**

reduction(operator : variable_list)

- **Operator (op) examples**
  - + sum
  - * product
  - max maximum
  - min minimum
  - & | ^ bitwise reductions
  - && || logical reductions
  - - allowed, but usually a bit confusing
- **Variable list**
  - One or more **scalar variables**:
    - reduction(+:sum)
    - reduction(max:maxVal)
    - reduction(+:sum, count)

Multiple Reductions in One Loop

- **Why this is useful**
  - One pass over the array
  - No race conditions
  - Fast and scalable

```
long long sum = 0;
int maxVal = INT_MIN;

#pragma omp parallel for reduction(+:sum) reduction(max:maxVal)
for (int i = 0; i < N; i++) {
    sum += a[i];
    if (a[i] > maxVal) maxVal = a[i];
}
```

Common Mistakes (What NOT to Do)

- **Race condition (no reduction):**
  - #pragma omp parallel for

```
for (int i = 0; i < N; i++)
    sum += a[i];
```

- **Wrong: reduction on an expression**
  - reduction(+:sum + a[i]) // invalid
- **Wrong: reduction on an array element**
  - reduction(+:sum[i]) // invalid

Exercise about Reduction

Find the **maximum value** in an array

max_array.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <limits.h>

#define N 1000000

int main(void) {
    int *a = malloc(N * sizeof(int));
    if (!a) return 1;

    // Fill array with some values
    for (int i = 0; i < N; i++)
        a[i] = i % 10000;   // values from 0..9999

    int maxVal = INT_MIN;

    #pragma omp parallel for reduction(max:maxVal)
    for (int i = 0; i < N; i++) {
        if (a[i] > maxVal)
            maxVal = a[i];
    }

    printf("Max value = %d (expected 9999)\n", maxVal);

    free(a);
    return 0;
}

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./max_array
Max value = 9999 (expected 9999)
h8wcd@fcs-0028:~/openmp-demo$
```

Solution 3 – The Critical Section

- The "Traffic Light" (#pragma omp critical)
  - **What it does:** It creates a "Critical Section" or a guarded zone.
  - **The Rule:** Only **one** thread is allowed to execute the code inside the { ... } block at a time.
  - **Mechanism:** If Thread A is inside the critical section, Thread B must **wait** (sleep/spin) at the entrance until Thread A leaves.
  - **Analogy:** One-marker whiteboard with 20 students working in parallel on a problem
  - **Critical vs. Atomic (The Key Distinction):**
    - **Atomic:** Optimized for **single instructions** (like x += 1). It often uses special fast hardware capabilities.
    - **Critical:** A general lock for **blocks of code**. It is slower but can handle complex logic (e.g., writing to a file, updating a linked list, or calling a function).

critical_parallel.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000

int main(void) {
    long long sum = 0;
    int *a = (int *)malloc((size_t)N * sizeof(int));
    if (!a) {
        perror("malloc failed");
        return 1;
    }

    // Initialize array with 1s. Correct sum should be N.
    for (int i = 0; i < N; i++) a[i] = 1;

    // Optional: set a fixed number of threads for a consistent demo
    omp_set_num_threads(4);

    // Automatic work-sharing + critical section (correct but often slow)
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        #pragma omp critical
        {
            sum += a[i];
        }
    }

    printf("Critical Sum: %lld (Expected: %d)\n", sum, N);
    if (sum != N) printf("FAILURE: Wrong result (should not happen with critical)!\n");

    free(a);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ gcc -fopenmp critical_parallel.c -o critical_parallel
h8wcd@fcs-0028:~/openmp-demo$ ./critical_parallel
Critical Sum: 1000000 (Expected: 1000000)
h8wcd@fcs-0028:~/openmp-demo$
```

Where critical is actually necessary

- You cannot use atomic for printf.
- If you want to print clean messages without them getting mixed up mid-sentence, critical is the only choice.

```
#pragma omp critical
{
    // Atomic cannot handle this!
    printf("Thread %d is processing item %d\n", omp_get_thread_num(), i);
    write_to_log_file("Log entry...");
}
```

The Cost of Safety (Performance Hierarchy)

- Performance: Choosing the Right Lock
  - Not all thread-safety tools cost the same.
    - **reduction (Fastest):** No locking at all during the loop. Threads work entirely on private memory. Merging happens once at the very end.
      - *Speed:* **100% (Near theoretical max)**
    - **atomic (Medium):** Locks only the specific memory address for one CPU instruction. Modern hardware handles this efficiently.
      - *Speed:* **~10-50% slower** than reduction (depending on contention).
    - **critical (Slowest):** A general-purpose "mutex" lock. It forces the operating system to intervene, potentially putting threads to sleep if they have to wait.
      - *Speed:* **~100x slower**. It effectively turns your parallel code into serial code (or worse, due to overhead).

The Speed Test Code

speed_test.c

```c
/* speed_test.c */
#include <stdio.h>
#include <omp.h>

#define N 100000000 // 100 Million to make the delay visible

int main() {
    long long sum;
    double start, end;
    int a_val = 1; // We assume a[i] is always 1 for simplicity

    printf("Running Speed Test with N = %d...\n", N);
    printf("------------------------------------------------\n");

    // --- TEST 1: REDUCTION (The Gold Standard) ---
    sum = 0;
    start = omp_get_wtime();

    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < N; i++) {
        sum += a_val;
    }

    end = omp_get_wtime();
    printf("1. Reduction Time: %f seconds (Sum: %lld)\n", end - start, sum);

    // --- TEST 2: ATOMIC (The Hardware Lock) ---
    sum = 0;
    start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        #pragma omp atomic
        sum += a_val;
    }

    end = omp_get_wtime();
    printf("2. Atomic Time:    %f seconds (Sum: %lld)\n", end - start, sum);


    // --- TEST 3: CRITICAL (The Sledgehammer) ---
    // We reduce N for this test because CRITICAL is SO slow
    // that waiting for 100 million iterations might hang your class demo!
    int N_small = N / 100; // Run only 1% of the workload
    sum = 0;

    start = omp_get_wtime();

    #pragma omp parallel for
    for (int i = 0; i < N_small; i++) {
        #pragma omp critical
        {
            sum += a_val;
        }
    }
```

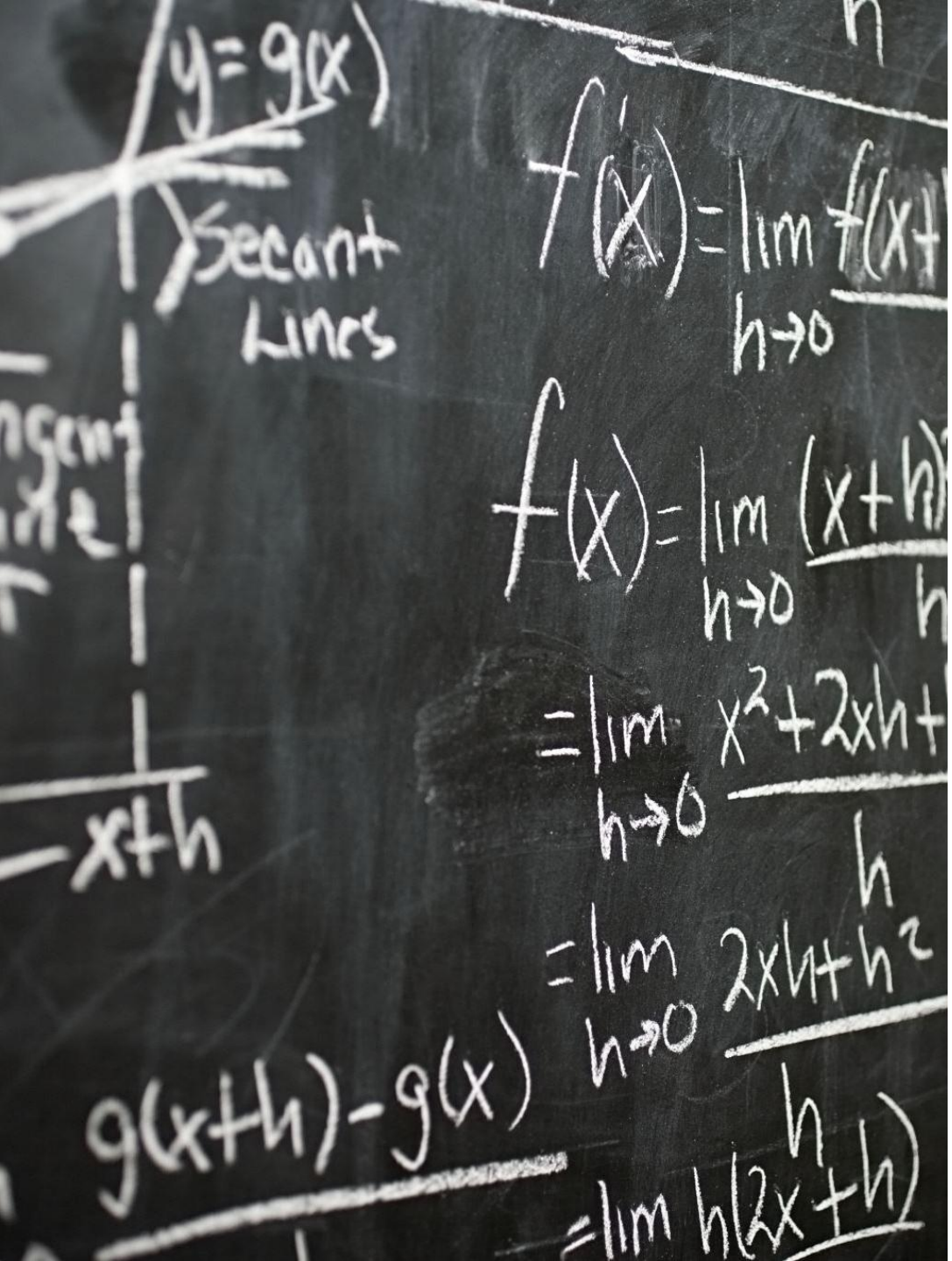
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
h8wcd@fcs-0028:~/openmp-demo$ ./speed_test
Running Speed Test with N = 100000000...
------------------------------------------------
1. Reduction Time: 0.030391 seconds (Sum: 100000000)
2. Atomic Time:    2.995461 seconds (Sum: 100000000)
3. Critical Time:  19.700789 seconds (Estimated for full N)
h8wcd@fcs-0028:~/openmp-demo$
```

Activity 1

- You have sensor readings in an array a[] (size N). You want to compute:
- **Total energy** = sum of all readings
- **Report the maximum reading** and print **exactly one line** summarizing it (avoid messy interleaved output)
- You will parallelize the work with OpenMP.
- **Rules**
  - You may only use these OpenMP constructs/clauses:
    parallel, for, reduction, critical, atomic
    (No single, no tasks, no sections, no locks, etc.)
  - Use **exactly 4 threads**.
  - The program must be correct for any N ≥ 1.

Work Sharing Loops



Tired of Manual Math?

- Previously, we calculated start = id * chunk.
- It's error-prone.
- OpenMP automates this.

Scheduling - Who does what?

- How does OpenMP assign i to threads?
  - Default is usually "Static", but we can control it.

OpenMP Scheduling (Why it matters)

- **Goal:** distribute loop iterations across threads efficiently.
- When you write:

```
#pragma omp parallel
{
  #pragma omp for
  for (int i = 0; i < N; i++) work(i);
}
```

OpenMP must decide:

  - Which thread executes which iterations?
  - How to avoid idle threads?
  - How to reduce overhead?
- **Scheduling = the policy used to assign loop iterations to threads.**

The 3 common options

- You will see these most often:
  - **Default schedule**
    - #pragma omp for
  - **Static schedule**
    - #pragma omp for schedule(static)
  - **Dynamic schedule**
    - #pragma omp for schedule(dynamic)
  - You can also control chunk size:
    - schedule(static, chunk)
    - schedule(dynamic, chunk)

Default schedule (#pragma omp for)

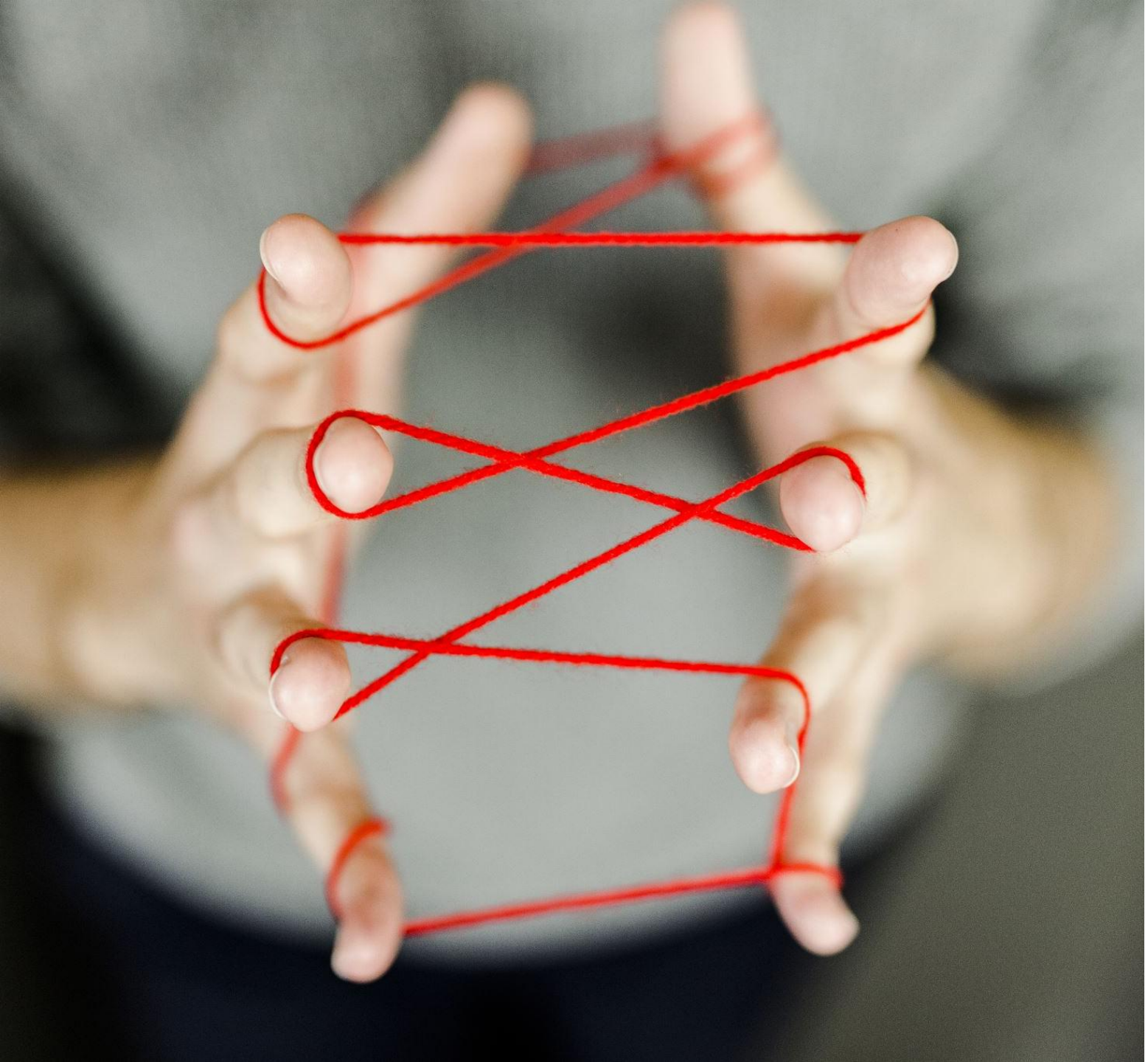
```
#pragma omp for
for (int i = 0; i < N; i++) work(i);
```

- Meaning: OpenMP uses the default schedule (often static).
- Pros
  - simplest code
  - usually good when work is uniform
  - low overhead
- Cons
  - may be bad if some iterations take much longer than others
- Key point: “No schedule” does NOT mean “OpenMP magically finds the best every time”.

The omp for Directive

- Tells OpenMP to distribute loop iterations among existing threads.

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < N; i++) {
        a[i] = b[i] + c[i];
    }
}
```



Requirements for Parallel Loops

1. **Canonical Shape:** Must be for(i=start; i<end; i++). No while loops.
2. **No Loop Dependencies:** Iteration i cannot depend on i-1.
3. **No Break:** You cannot break out of a parallel loop.

Static scheduling (schedule(static))

```
#pragma omp for schedule(static)
for (int i = 0; i < N; i++) work(i);
```

- Meaning: OpenMP divides iterations once at the start into fixed blocks.
- Example with 4 threads and N=16 (default chunking):
  - T0: i = 0..3
  - T1: i = 4..7
  - T2: i = 8..11
  - T3: i = 12..15
- Pros
  - very low overhead (fast)
  - good cache locality (each thread works on a continuous block)
- Cons
  - can cause load imbalance if work per iteration is uneven

Static is GREAT (Example)

- **Example: uniform work per iteration**

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) {
    c[i] = a[i] * b[i] + 3;
}
```

- Why static is best here:
  - every iteration does about the same amount of work
  - no thread gets “harder” iterations
  - minimal overhead

Static is BAD (Anti-example)

- **Anti-example: uneven work**

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) {
    if (i < N/4) heavy_work();   // heavy work clustered early
    else light_work();
}
```

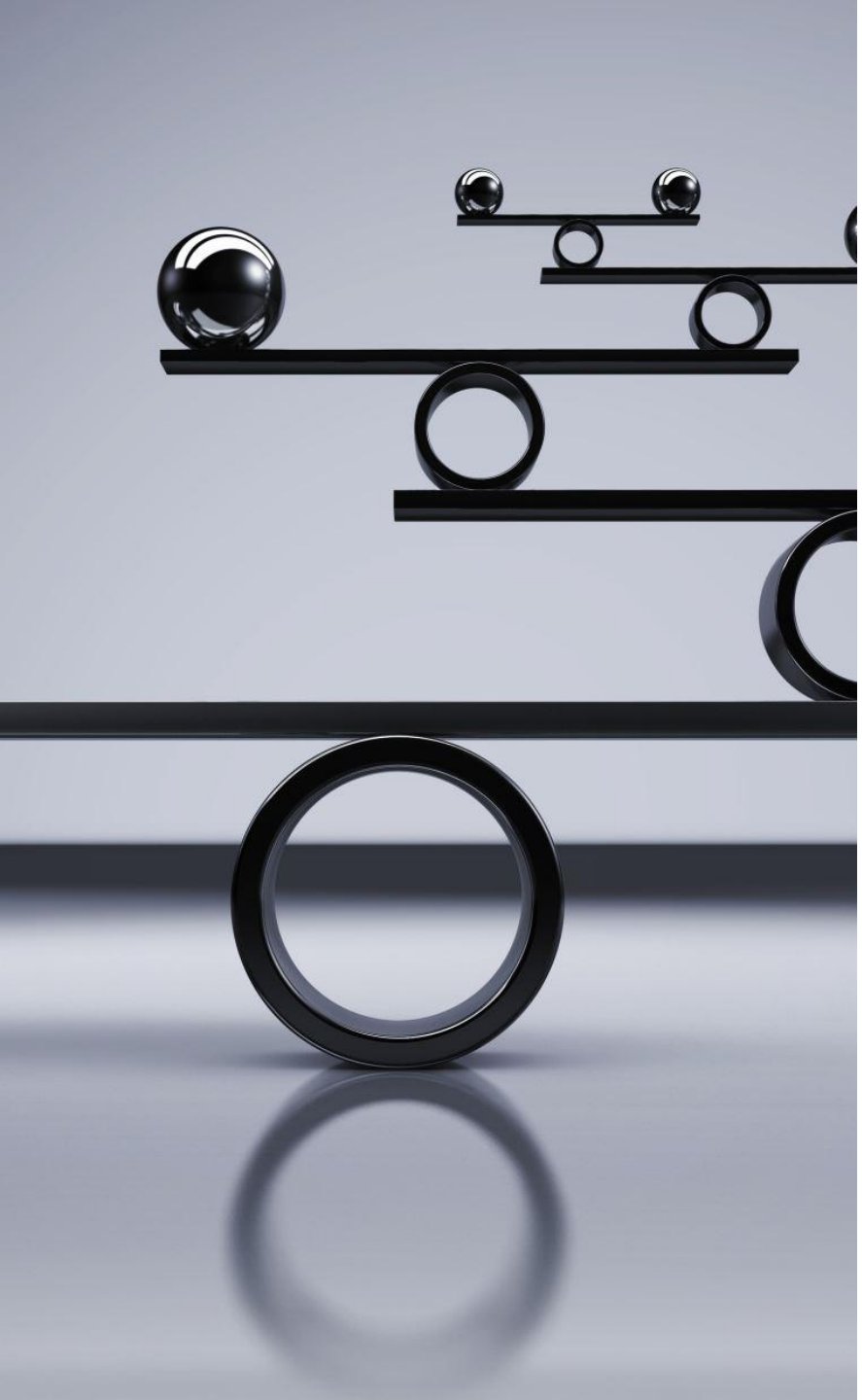
What can happen:

  - one thread gets many “heavy” iterations → becomes the bottleneck
  - other threads finish early → wait idle

- **Static causes load imbalance**

The Problem – Load Imbalance

- When "Fair" Isn't "Fast"
  - OpenMP's default is **Static Scheduling**. It divides loop iterations evenly (e.g., Thread 0 gets 0–49, Thread 1 gets 50–99).
  - **The Trap:** What if iterations 50–99 take 10x longer to process than 0–49?
    - *Example:* Processing a triangular matrix or an image that is mostly empty at the top but complex at the bottom.
  - **The Result:** Thread 0 finishes in 1 second and sits idle. Thread 1 works for 10 seconds. The total time is 10 seconds.
  - **The Goal:** Keep all threads busy. If Thread 0 finishes early, it should help Thread 1!



Strategy B (Dynamic)

- **Scenario:** Assignments are in a central pile.
- Each TA takes 5 assignments. Grades them. comes back for 5 more.
- *Pro:* Load balancing. If you are slow, you take fewer tasks.
- *Con:* Overhead (Walking to the pile).

Dynamic scheduling (schedule(dynamic))

```
#pragma omp for schedule(dynamic)
for (int i = 0; i < N; i++) work(i);
```

- Meaning: iterations are assigned gradually:
  - each thread grabs a chunk
  - when done, it grabs another
- Pros
  - excellent load balancing when work is unpredictable
  - avoids threads sitting idle
- Cons
  - higher overhead (extra coordination)
  - worse cache locality (threads jump around)

Dynamic is GREAT (Example)

- **Example: variable iteration cost**

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < N; i++) {
    solve_problem_instance(i); // runtime varies per i
}
```

- Why dynamic helps:
  - fast threads automatically do more iterations
  - slow threads do fewer iterations
  - better overall finish time
- **Recommended:** dynamic (or guided)

Dynamic is BAD (Anti-example)

- **Anti-example: tiny uniform work**

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < N; i++) {
    c[i] = a[i] + 1;
}
```

- Problem:
  - each iteration is extremely fast
  - dynamic scheduling overhead becomes comparable to the work
  - performance can be worse than static (and even worse than serial)
- **Dynamic wastes time on scheduling**

Guided scheduling (schedule(guided))

- guided is like dynamic, but with a smart twist:
  - At the start, threads get **large chunks** (low overhead).
  - As the loop progresses, chunk sizes **shrink**, so the remaining work gets balanced better near the end.
- Syntax:

#pragma omp for schedule(guided)

  - or with a minimum chunk size:

#pragma omp for schedule(guided, 64)

- **Why it exists:**
  - Dynamic gives great balance but can have high overhead.
  - Guided aims for **almost dynamic's balance** with **less overhead**.
- **When it's good:**
  - Uneven workloads
  - Large loops
  - You want better balance than static without fully paying dynamic's overhead

```
void work(int i) {
    // simulate variable work: larger i => more work
    for (volatile int k = 0; k < i * 50; k++) { }
}

#pragma omp parallel for schedule(guided, 64)
for (int i = 1; i <= 10000; i++) {
    work(i);
}
```

Chunk size (static and dynamic)

- You can control chunk size:

#pragma omp for schedule(static, 4)

- Meaning: each thread gets chunks of 4 iterations at a time.

#pragma omp for schedule(dynamic, 10)

- Meaning: threads request 10 iterations per chunk.
- **Tradeoff**
  - Smaller chunk → better load balance, more overhead
  - Larger chunk → less overhead, worse balance

What is the goal of setting chunk size in static?

- Chunk size controls the **granularity** of the distribution.

#pragma omp for schedule(static, chunk)

- Instead of giving each thread one big block, OpenMP gives **blocks of chunk iterations**, assigned in a round-robin pattern.
- Example: N=16, 4 threads, schedule(static,2):
  - T0: 0–1, 8–9
  - T1: 2–3, 10–11
  - T2: 4–5, 12–13
  - T3: 6–7, 14–15
- **Why do this?**
  - To **reduce load imbalance** when some iterations are heavier than others, while keeping low overhead compared to dynamic.
  - It's a compromise: mostly static (cheap), but with better balancing.
- **Trade-off:**
  - Smaller chunk → better balance, slightly worse locality
  - Larger chunk → better locality, potentially worse balance

Summary table (what to choose?)

- **If work per iteration is…**
  - **Uniform and predictable**
    - default or schedule(static)
    - best performance, low overhead
  - **Uneven or unpredictable**
    - schedule(dynamic) (or guided)
    - best load balancing
  - **Very small loop body**
    - avoid dynamic
    - scheduling overhead dominates

Rule of thumb

- **Start simple:**

#pragma omp parallel for

- Then ask:
  - Are all iterations similar cost?
    - Yes → keep default/static
  - Some iterations much slower?
    - Use dynamic (+ chunk)
  - Too much overhead?
    - increase chunk size or switch to static

Analogy: Grading Assignments

- **Scenario:** 100 assignments, 4 TAs.
- **Strategy A (Static):** I give a pile of 25 to each TA.
  - *Pro:* Easy to hand out.
  - *Con:* What if TA #1 has easy assignments and TA #2 has illegible code? TA #1 finishes early and idles.

The Solutions (Static vs. Dynamic)

- **1. Static (schedule(static, chunk))**
  - **Logic:** "Deal the cards." Assignments are fixed before the loop starts.
  - **Pros:** Lowest overhead (fastest for simple arrays).
  - **Cons:** Cannot handle unpredictable workloads.
  - **Analogy:** Assigned seats in an exam.
- **2. Dynamic (schedule(dynamic, chunk))**
  - **Logic:** "First come, first served." Threads grab a chunk of iterations, process them, then come back for more.
  - **Pros:** Perfect load balancing for uneven work.
  - **Cons:** Higher overhead (management cost).
  - **Analogy:** A "Take a Number" system at a deli.
- **3. Guided (schedule(guided, chunk))**
  - **Logic:** Starts with large chunks to reduce overhead, then exponentially shrinks chunk size to smooth out the end.
  - **Verdict:** Often the best compromise.

Example

schedule_viewer.c

```c
/* schedule_viewer.c */
#include <stdio.h>
#include <omp.h>
#include <unistd.h> // for usleep

#define N 20

int main() {
    omp_set_num_threads(4);

    printf("--- STATIC SCHEDULE (Chunk 2) ---\n");
    // "Deal cards" in blocks of 2:
    // T0: [0,1], T1: [2,3], T2: [4,5], T3: [6,7], T0: [8,9]...
    #pragma omp parallel for schedule(static, 2)
    for (int i = 0; i < N; i++) {
        usleep(1000); // Tiny delay to prevent print collisions
        printf("Iter %02d processed by Thread %d\n", i, omp_get_thread_num());
    }

    printf("\n--- DYNAMIC SCHEDULE (Chunk 2) ---\n");
    // "Grab bag": Whoever is free grabs the next 2 iterations.
    // Order is non-deterministic.
    #pragma omp parallel for schedule(dynamic, 2)
    for (int i = 0; i < N; i++) {
        usleep(1000);
        printf("Iter %02d processed by Thread %d\n", i, omp_get_thread_num());
    }

    return 0;
}
```

```
h8wcd@fcs-0028:~/openmp-demo$ ./schedule_viewer
Iter 06 processed by Thread 3
Iter 04 processed by Thread 2
Iter 02 processed by Thread 1
Iter 00 processed by Thread 0
Iter 07 processed by Thread 3
Iter 05 processed by Thread 2
Iter 03 processed by Thread 1
Iter 01 processed by Thread 0
Iter 14 processed by Thread 3
Iter 12 processed by Thread 2
Iter 10 processed by Thread 1
Iter 08 processed by Thread 0
Iter 13 processed by Thread 2
Iter 15 processed by Thread 3
Iter 11 processed by Thread 1
Iter 09 processed by Thread 0
Iter 18 processed by Thread 1
Iter 16 processed by Thread 0
Iter 19 processed by Thread 1
Iter 17 processed by Thread 0

--- DYNAMIC SCHEDULE (Chunk 2) ---
Iter 04 processed by Thread 0
Iter 00 processed by Thread 1
Iter 02 processed by Thread 3
Iter 06 processed by Thread 2
Iter 05 processed by Thread 0
Iter 01 processed by Thread 1
Iter 03 processed by Thread 3
Iter 07 processed by Thread 2
Iter 10 processed by Thread 1
Iter 12 processed by Thread 3
Iter 08 processed by Thread 0
Iter 14 processed by Thread 2
Iter 11 processed by Thread 1
Iter 15 processed by Thread 2
Iter 09 processed by Thread 0
Iter 13 processed by Thread 3
Iter 16 processed by Thread 1
Iter 18 processed by Thread 2
Iter 17 processed by Thread 1
Iter 19 processed by Thread 2
h8wcd@fcs-0028:~/openmp-demo$
```

Explain the example

schedule_viewer.c

```c
/* schedule_viewer.c */
#include <stdio.h>
#include <omp.h>
#include <unistd.h> // for usleep

#define N 20

int main() {
    omp_set_num_threads(4);

    printf("--- STATIC SCHEDULE (Chunk 2) ---\n");
    // "Deal cards" in blocks of 2:
    // T0: [0,1], T1: [2,3], T2: [4,5], T3: [6,7], T0: [8,9]...
    #pragma omp parallel for schedule(static, 2)
    for (int i = 0; i < N; i++) {
        usleep(1000); // Tiny delay to prevent print collisions
        printf("Iter %02d processed by Thread %d\n", i, omp_get_thread_num());
    }

    printf("\n--- DYNAMIC SCHEDULE (Chunk 2) ---\n");
    // "Grab bag": Whoever is free grabs the next 2 iterations.
    // Order is non-deterministic.
    #pragma omp parallel for schedule(dynamic, 2)
    for (int i = 0; i < N; i++) {
        usleep(1000);
        printf("Iter %02d processed by Thread %d\n", i, omp_get_thread_num());
    }

    return 0;
}
```

- **Meaning of static,2**
  - Divide the loop into **chunks of 2 iterations**
  - Assign these chunks to threads **before execution starts**
- It is **deterministic** (same assignment every run)
- **Chunks for N=20:**
  - The iterations are grouped like this:
    - chunk0: (0,1)
    - chunk1: (2,3)
    - chunk2: (4,5)
    - chunk3: (6,7)
    - chunk4: (8,9)
    - chunk5: (10,11)
    - chunk6: (12,13)
    - chunk7: (14,15)
    - chunk8: (16,17)
    - chunk9: (18,19)
  - Now OpenMP distributes chunks to threads in a **round-robin** style:
    - **Thread 0** gets chunks 0,4,8 → (0,1), (8,9), (16,17)
    - **Thread 1** gets chunks 1,5,9 → (2,3), (10,11), (18,19)
    - **Thread 2** gets chunks 2,6 → (4,5), (12,13)
    - **Thread 3** gets chunks 3,7 → (6,7), (14,15)

Explain the example

C schedule_viewer.c

```c
/* schedule_viewer.c */
#include <stdio.h>
#include <omp.h>
#include <unistd.h> // for usleep

#define N 20

int main() {
    omp_set_num_threads(4);

    printf("--- STATIC SCHEDULE (Chunk 2) ---\n");
    // "Deal cards" in blocks of 2:
    // T0: [0,1], T1: [2,3], T2: [4,5], T3: [6,7], T0: [8,9]...
    #pragma omp parallel for schedule(static, 2)
    for (int i = 0; i < N; i++) {
        usleep(1000); // Tiny delay to prevent print collisions
        printf("Iter %02d processed by Thread %d\n", i, omp_get_thread_num());
    }

    printf("\n--- DYNAMIC SCHEDULE (Chunk 2) ---\n");
    // "Grab bag": Whoever is free grabs the next 2 iterations.
    // Order is non-deterministic.
    #pragma omp parallel for schedule(dynamic, 2)
    for (int i = 0; i < N; i++) {
        usleep(1000);
        printf("Iter %02d processed by Thread %d\n", i, omp_get_thread_num());
    }

    return 0;
}
```

- **Meaning of dynamic,2**
  - Still break the loop into **chunks of 2**
  - BUT: chunks are **not assigned ahead of time**
  - Instead: when a thread finishes its chunk, it asks for the **next available chunk**
- **What happens at runtime**
  - At the start:
    - 4 threads each grab one chunk:
      - Example:
        - T0 gets (0,1)
        - T1 gets (2,3)
        - T2 gets (4,5)
        - T3 gets (6,7)
  - Then whichever thread finishes first grabs the next chunk:
    - next chunk is (8,9)
    - then (10,11)
    - etc.
- **Why the assignment changes between runs**
  - Because the “who finishes first” depends on small timing differences.
  - Even tiny differences (cache, OS time slicing, etc.) can change which thread becomes free first.
- Dynamic = better load balance when work per iteration is uneven
  Dynamic = more overhead + less predictable iteration ownership